JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class AbstractQueuedSynchronizer

java.lang.Object
    java.util.concurrent.locks.AbstractOwnableSynchronizer
        java.util.concurrent.locks.AbstractQueuedSynchronizer

All Implemented Interfaces:

Serializable

```
public abstract class AbstractQueuedSynchronizer
extends AbstractOwnableSynchronizer
implements Serializable
```

Provides a framework for implementing blocking locks and related synchronizers (semaphores, events, etc) that rely on first-in-first-out (FIFO) wait queues. This class is designed to be a useful basis for most kinds of synchronizers that rely on a single atomic `int` value to represent state. Subclasses must define the protected methods that change this state, and which define what that state means in terms of this object being acquired or released. Given these, the other methods in this class carry out all queuing and blocking mechanics. Subclasses can maintain other state fields, but only the atomically updated `int` value manipulated using methods `getState()`, `setState(int)` and `compareAndSetState(int, int)` is tracked with respect to synchronization.

Subclasses should be defined as non-public internal helper classes that are used to implement the synchronization properties of their enclosing class. Class `AbstractQueuedSynchronizer` does not implement any synchronization interface. Instead it defines methods such as `acquireInterruptibly(int)` that can be invoked as appropriate by concrete locks and related synchronizers to implement their public methods.

This class supports either or both a default *exclusive* mode and a *shared* mode. When acquired in exclusive mode, attempted acquires by other threads cannot succeed. Shared mode acquires by multiple threads may (but need not) succeed. This class does not "understand" these differences except in the mechanical sense that when a shared mode acquire succeeds, the next waiting thread (if one exists) must also determine whether it can acquire as well. Threads waiting in the different modes share the same FIFO queue. Usually, implementation subclasses support only one of these modes, but both can come into play for example in a `ReadWriteLock`. Subclasses that support only exclusive or only shared modes need not define the methods supporting the unused mode.

This class defines a nested `AbstractQueuedSynchronizer.ConditionObject` class that can be used as a `Condition` implementation by subclasses supporting exclusive mode for which method `isHeldExclusively()` reports

whether synchronization is exclusively held with respect to the current thread, method `release(int)` invoked with the current `getState()` value fully releases this object, and the underlying version of `acquire(int)`, given this saved state value, eventually restores this object to its previous acquired state. No `AbstractQueuedSynchronizer` method otherwise creates such a condition, so if this constraint cannot be met, do not use it. The behavior of `AbstractQueuedSynchronizer.ConditionObject` depends of course on the semantics of its synchronizer implementation.

This class provides inspection, instrumentation, and monitoring methods for the internal queue, as well as similar methods for condition objects. These can be exported as desired into classes using an `AbstractQueuedSynchronizer` for their synchronization mechanics.

Serialization of this class stores only the underlying atomic integer maintaining state, so deserialized objects have empty thread queues. Typical subclasses requiring serializability will define a `readObject` method that restores this to a known initial state upon deserialization.

Usage

To use this class as the basis of a synchronizer, redefine the following methods, as applicable, by inspecting and/or modifying the synchronization state using `getState()`, `setState(int)` and/or `compareAndSetState(int, int)`:

- `tryAcquire(int)`
- `tryRelease(int)`
- `tryAcquireShared(int)`
- `tryReleaseShared(int)`
- `isHeldExclusively()`

Each of these methods by default throws `UnsupportedOperationException`. Implementations of these methods must be internally thread-safe, and should in general be short and not block. Defining these methods is the *only* supported means of using this class. All other methods are declared `final` because they cannot be independently varied.

You may also find the inherited methods from `AbstractOwnableSynchronizer` useful to keep track of the thread owning an exclusive synchronizer. You are encouraged to use them -- this enables monitoring and diagnostic tools to assist users in determining which threads hold locks.

Even though this class is based on an internal FIFO queue, it does not automatically enforce FIFO acquisition policies. The core of exclusive synchronization takes the form:

```
Acquire:
    while (!tryAcquire(arg)) {
       enqueue thread if it is not already queued;
       possibly block current thread;
    }

Release:
    if (tryRelease(arg))
       unblock the first queued thread;
```

(Shared mode is similar but may involve cascading signals.)

Because checks in acquire are invoked before enqueuing, a newly acquiring thread may *barge* ahead of others that are blocked and queued. However, you can, if desired, define `tryAcquire` and/or `tryAcquireShared` to disable barging by internally invoking one or more of the inspection methods, thereby providing a *fair* FIFO acquisition order. In particular, most fair synchronizers can define `tryAcquire` to return `false` if `hasQueuedPredecessors()` (a method specifically designed to be used by fair synchronizers) returns `true`. Other variations are possible.

Throughput and scalability are generally highest for the default barging (also known as *greedy*, *renouncement*, and *convoy-avoidance*) strategy. While this is not guaranteed to be fair or starvation-free, earlier queued threads are allowed to recontend before later queued threads, and each recontention has an unbiased chance to succeed against incoming threads. Also, while acquires do not "spin" in the usual sense, they may perform multiple invocations of `tryAcquire` interspersed with other computations before blocking. This gives most of the benefits of spins when exclusive synchronization is only briefly held, without most of the liabilities when it isn't. If so desired, you can augment this by preceding calls to acquire methods with "fast-path" checks, possibly prechecking `hasContended()` and/or `hasQueuedThreads()` to only do so if the synchronizer is likely not to be contended.

This class provides an efficient and scalable basis for synchronization in part by specializing its range of use to synchronizers that can rely on `int` state, acquire, and release parameters, and an internal FIFO wait queue. When this does not suffice, you can build synchronizers from a lower level using `atomic` classes, your own custom `Queue` classes, and `LockSupport` blocking support.

Usage Examples

Here is a non-reentrant mutual exclusion lock class that uses the value zero to represent the unlocked state, and one to represent the locked state. While a non-reentrant lock does not strictly require recording of the current owner

thread, this class does so anyway to make usage easier to monitor. It also supports conditions and exposes some instrumentation methods:

```
class Mutex implements Lock, java.io.Serializable {

  // Our internal helper class
  private static class Sync extends AbstractQueuedSynchronizer {
    // Acquires the lock if state is zero
    public boolean tryAcquire(int acquires) {
      assert acquires == 1; // Otherwise unused
      if (compareAndSetState(0, 1)) {
        setExclusiveOwnerThread(Thread.currentThread());
        return true;
      }
      return false;
    }

    // Releases the lock by setting state to zero
    protected boolean tryRelease(int releases) {
      assert releases == 1; // Otherwise unused
      if (!isHeldExclusively())
        throw new IllegalMonitorStateException();
      setExclusiveOwnerThread(null);
      setState(0);
      return true;
    }

    // Reports whether in locked state
    public boolean isLocked() {
      return getState() != 0;
    }

    public boolean isHeldExclusively() {
      // a data race, but safe due to out-of-thin-air guarantees
      return getExclusiveOwnerThread() == Thread.currentThread();
    }

    // Provides a Condition
    public Condition newCondition() {
      return new ConditionObject();
    }

    // Deserializes properly
```

```
    private void readObject(ObjectInputStream s)
        throws IOException, ClassNotFoundException {
      s.defaultReadObject();
      setState(0); // reset to unlocked state
    }
  }

  // The sync object does all the hard work. We just forward to it.
  private final Sync sync = new Sync();

  public void lock()              { sync.acquire(1); }
  public boolean tryLock()        { return sync.tryAcquire(1); }
  public void unlock()            { sync.release(1); }
  public Condition newCondition() { return sync.newCondition(); }
  public boolean isLocked()       { return sync.isLocked(); }
  public boolean isHeldByCurrentThread() {
    return sync.isHeldExclusively();
  }
  public boolean hasQueuedThreads() {
    return sync.hasQueuedThreads();
  }
  public void lockInterruptibly() throws InterruptedException {
    sync.acquireInterruptibly(1);
  }
  public boolean tryLock(long timeout, TimeUnit unit)
      throws InterruptedException {
    return sync.tryAcquireNanos(1, unit.toNanos(timeout));
  }
}
```

Here is a latch class that is like a `CountDownLatch` except that it only requires a single `signal` to fire. Because a latch is non-exclusive, it uses the `shared` acquire and release methods.

```
class BooleanLatch {

  private static class Sync extends AbstractQueuedSynchronizer {
    boolean isSignalled() { return getState() != 0; }

    protected int tryAcquireShared(int ignore) {
      return isSignalled() ? 1 : -1;
    }
```

```
    protected boolean tryReleaseShared(int ignore) {
      setState(1);
      return true;
    }
  }

  private final Sync sync = new Sync();
  public boolean isSignalled() { return sync.isSignalled(); }
  public void signal()         { sync.releaseShared(1); }
  public void await() throws InterruptedException {
    sync.acquireSharedInterruptibly(1);
  }
}
```

Since:

1.5

See Also:

Serialized Form

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description
class	`AbstractQueuedSynchroniz`	Condition implementation for a `AbstractQueuedSynchronizer` serving as the basis of a `Lock` implementation.

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	`AbstractQueuedSynchroniz`	Creates a new `AbstractQueuedSynchronizer` instance with initial synchronization state of zero.

Method Summary

All Methods	Instance Methods	

Modifier and Type	Method	Description
final void	acquire(int arg)	Acquires in exclusive mode, ignoring interrupts.
final void	acquireInterruptibly (int arg)	Acquires in exclusive mode, aborting if interrupted.
final void	acquireShared(int arg)	Acquires in shared mode, ignoring interrupts.
final void	acquireSharedInterruptib (int arg)	Acquires in shared mode, aborting if interrupted.
protected final boolean	compareAndSetState (int expect, int update)	Atomically sets synchronization state to the given updated value if the current state value equals the expected value.
final Collection <Thread>	getExclusiveQueuedThread	Returns a collection containing threads that may be waiting to acquire in exclusive mode.
final Thread	getFirstQueuedThread()	Returns the first (longest-waiting) thread in the queue, or null if no threads are currently queued.
final Collection <Thread>	getQueuedThreads()	Returns a collection containing threads that may be waiting to acquire.
final int	getQueueLength()	

		Returns an estimate of the number of threads waiting to acquire.
final Collection<Thread>	**getSharedQueuedThreads**()	Returns a collection containing threads that may be waiting to acquire in shared mode.
protected final int	**getState**()	Returns the current value of synchronization state.
final Collection<Thread>	**getWaitingThreads**(AbstractQueuedSynchroni	Returns a collection containing those threads that may be waiting on the given condition associated with this synchronizer.
final int	**getWaitQueueLength**(AbstractQueuedSynchroni	Returns an estimate of the number of threads waiting on the given condition associated with this synchronizer.
final boolean	**hasContended**()	Queries whether any threads have ever contended to acquire this synchronizer; that is, if an acquire method has ever blocked.
final boolean	**hasQueuedPredecessors**()	Queries whether any threads have been waiting to acquire longer than the current thread.
final boolean	**hasQueuedThreads**()	Queries whether any threads are waiting to acquire.
final boolean	**hasWaiters**(AbstractQueuedSynchroni	Queries whether any threads are waiting on the given condition

		associated with this synchronizer.
protected boolean	**isHeldExclusively**()	Returns true if synchronization is held exclusively with respect to the current (calling) thread.
final boolean	**isQueued**(Thread thread)	Returns true if the given thread is currently queued.
final boolean	**owns**(AbstractQueuedSynchroni	Queries whether the given ConditionObject uses this synchronizer as its lock.
final boolean	**release**(int arg)	Releases in exclusive mode.
final boolean	**releaseShared**(int arg)	Releases in shared mode.
protected final void	**setState**(int newState)	Sets the value of synchronization state.
String	**toString**()	Returns a string identifying this synchronizer, as well as its state.
protected boolean	**tryAcquire**(int arg)	Attempts to acquire in exclusive mode.
final boolean	**tryAcquireNanos**(int arg, long nanosTimeout)	Attempts to acquire in exclusive mode, aborting if interrupted, and failing if the given timeout elapses.
protected int	**tryAcquireShared**(int arg)	Attempts to acquire in shared mode.

final boolean	**tryAcquireSharedNanos**(int arg, long nanosTimeout)	Attempts to acquire in shared mode, aborting if interrupted, and failing if the given timeout elapses.
protected boolean	**tryRelease**(int arg)	Attempts to set the state to reflect a release in exclusive mode.
protected boolean	**tryReleaseShared**(int arg)	Attempts to set the state to reflect a release in shared mode.

Methods declared in class AbstractOwnableSynchronizer

getExclusiveOwnerThread, setExclusiveOwnerThread

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Details

AbstractQueuedSynchronizer

```
protected AbstractQueuedSynchronizer()
```

Creates a new `AbstractQueuedSynchronizer` instance with initial synchronization state of zero.

Method Details

getState

```
protected final int getState()
```

Returns the current value of synchronization state. This operation has memory semantics of a `volatile` read.

Returns:

current state value

setState

```
protected final void setState(int newState)
```

Sets the value of synchronization state. This operation has memory semantics of a `volatile` write.

Parameters:

`newState` - the new state value

compareAndSetState

```
protected final boolean compareAndSetState(int expect,
                                           int update)
```

Atomically sets synchronization state to the given updated value if the current state value equals the expected value. This operation has memory semantics of a `volatile` read and write.

Parameters:

`expect` - the expected value

`update` - the new value

Returns:

`true` if successful. False return indicates that the actual value was not equal to the expected value.

tryAcquire

```
protected boolean tryAcquire(int arg)
```

Attempts to acquire in exclusive mode. This method should query if the state of the object permits it to be acquired in the exclusive mode, and if so to acquire it.

This method is always invoked by the thread performing acquire. If this method reports failure, the acquire method may queue the thread, if it is not already queued, until it is signalled by a release from some other thread. This can be used to implement method `Lock.tryLock()`.

The default implementation throws `UnsupportedOperationException`.

Parameters:

`arg` - the acquire argument. This value is always the one passed to an acquire method, or is the value saved on entry to a condition wait. The value is otherwise uninterpreted and can represent anything you like.

Returns:

`true` if successful. Upon success, this object has been acquired.

Throws:

`IllegalMonitorStateException` - if acquiring would place this synchronizer in an illegal state. This exception must be thrown in a consistent fashion for synchronization to work correctly.

`UnsupportedOperationException` - if exclusive mode is not supported

tryRelease

```
protected boolean tryRelease(int arg)
```

Attempts to set the state to reflect a release in exclusive mode.

This method is always invoked by the thread performing release.

The default implementation throws `UnsupportedOperationException`.

Parameters:

`arg` - the release argument. This value is always the one passed to a release method, or the current state value upon entry to a condition wait. The value is otherwise uninterpreted and can represent anything you like.

Returns:

`true` if this object is now in a fully released state, so that any waiting threads may attempt to acquire; and `false` otherwise.

Throws:

`IllegalMonitorStateException` - if releasing would place this synchronizer in an illegal state. This exception must be thrown in a consistent fashion for synchronization to work correctly.

`UnsupportedOperationException` - if exclusive mode is not supported

tryAcquireShared

```
protected int tryAcquireShared(int arg)
```

Attempts to acquire in shared mode. This method should query if the state of the
object permits it to be acquired in the shared mode, and if so to acquire it.

This method is always invoked by the thread performing acquire. If this method reports failure, the acquire method may queue the thread, if it is not already queued, until it is signalled by a release from some other thread.

The default implementation throws `UnsupportedOperationException`.

Parameters:

`arg` - the acquire argument. This value is always the one passed to an acquire method, or is the value saved on entry to a condition wait. The value is otherwise uninterpreted and can represent anything you like.

Returns:

a negative value on failure; zero if acquisition in shared mode succeeded but no subsequent shared-mode acquire can succeed; and a positive value if acquisition in shared mode succeeded and subsequent shared-mode acquires might also succeed, in which case a subsequent waiting thread must check availability. (Support for three different return values enables this method to be used in contexts where acquires only sometimes act exclusively.) Upon success, this object has been acquired.

Throws:

`IllegalMonitorStateException` - if acquiring would place this synchronizer in an illegal state. This exception must be thrown in a consistent fashion for synchronization to work correctly.

`UnsupportedOperationException` - if shared mode is not supported

tryReleaseShared

```
protected boolean tryReleaseShared(int arg)
```

Attempts to set the state to reflect a release in shared mode.

This method is always invoked by the thread performing release.

The default implementation throws `UnsupportedOperationException`.

Parameters:

`arg` - the release argument. This value is always the one passed to a release method, or the current state value upon entry to a condition wait. The value is otherwise uninterpreted and can represent anything you like.

Returns:

`true` if this release of shared mode may permit a waiting acquire (shared or exclusive) to succeed; and `false` otherwise

Throws:

`IllegalMonitorStateException` - if releasing would place this synchronizer in an illegal state. This exception must be thrown in a consistent fashion for synchronization to work correctly.

`UnsupportedOperationException` - if shared mode is not supported

isHeldExclusively

```
protected boolean isHeldExclusively()
```

Returns `true` if synchronization is held exclusively with respect to the current (calling) thread. This method is invoked upon each call to a `AbstractQueuedSynchronizer.ConditionObject` method.

The default implementation throws `UnsupportedOperationException`. This method is invoked internally only within `AbstractQueuedSynchronizer.ConditionObject` methods, so need not be defined if conditions are not used.

Returns:

`true` if synchronization is held exclusively; `false` otherwise

Throws:

`UnsupportedOperationException` - if conditions are not supported

acquire

```
public final void acquire(int arg)
```

Acquires in exclusive mode, ignoring interrupts. Implemented by invoking at least once `tryAcquire(int)`, returning on success. Otherwise the thread is queued, possibly repeatedly blocking and unblocking, invoking `tryAcquire(int)` until success. This method can be used to implement method `Lock.lock()`.

Parameters:

`arg` - the acquire argument. This value is conveyed to `tryAcquire(int)` but is otherwise uninterpreted and can represent anything you like.

acquireInterruptibly

```
public final void acquireInterruptibly(int arg)
                                throws InterruptedException
```

Acquires in exclusive mode, aborting if interrupted. Implemented by first checking interrupted status, then invoking at least once `tryAcquire(int)`, returning on success. Otherwise the thread is queued, possibly repeatedly blocking and unblocking, invoking `tryAcquire(int)` until success or the thread is interrupted. This method can be used to implement method `Lock.lockInterruptibly()`.

Parameters:

`arg` - the acquire argument. This value is conveyed to `tryAcquire(int)` but is otherwise uninterpreted and can represent anything you like.

Throws:

`InterruptedException` - if the current thread is interrupted

tryAcquireNanos

```
public final boolean tryAcquireNanos(int arg,
                                     long nanosTimeout)
                              throws InterruptedException
```

Attempts to acquire in exclusive mode, aborting if interrupted, and failing if the given timeout elapses. Implemented by first checking interrupted status, then invoking at least once `tryAcquire(int)`, returning on success. Otherwise, the thread is queued, possibly repeatedly blocking and unblocking, invoking `tryAcquire(int)` until success or the thread is interrupted or the timeout elapses. This method can be used to implement method `Lock.tryLock(long, TimeUnit)`.

Parameters:

`arg` - the acquire argument. This value is conveyed to `tryAcquire(int)` but is otherwise uninterpreted and can represent anything you like.

`nanosTimeout` - the maximum number of nanoseconds to wait

Returns:

`true` if acquired; `false` if timed out

Throws:

`InterruptedException` - if the current thread is interrupted

release

```
public final boolean release(int arg)
```

Releases in exclusive mode. Implemented by unblocking one or more threads if `tryRelease(int)` returns true. This method can be used to implement method `Lock.unlock()`.

Parameters:

`arg` - the release argument. This value is conveyed to `tryRelease(int)` but is otherwise uninterpreted and can represent anything you like.

Returns:

the value returned from `tryRelease(int)`

acquireShared

```
public final void acquireShared(int arg)
```

Acquires in shared mode, ignoring interrupts. Implemented by first invoking at least once `tryAcquireShared(int)`, returning on success. Otherwise the thread is queued, possibly repeatedly blocking and unblocking, invoking `tryAcquireShared(int)` until success.

Parameters:

`arg` - the acquire argument. This value is conveyed to `tryAcquireShared(int)` but is otherwise uninterpreted and can represent anything you like.

acquireSharedInterruptibly

```
public final void acquireSharedInterruptibly(int arg)
                                       throws InterruptedException
```

Acquires in shared mode, aborting if interrupted. Implemented by first checking interrupted status, then invoking at least once `tryAcquireShared(int)`, returning on success. Otherwise the thread is queued, possibly repeatedly blocking and unblocking, invoking `tryAcquireShared(int)` until success or the thread is interrupted.

Parameters:

`arg` - the acquire argument. This value is conveyed to `tryAcquireShared(int)` but is otherwise uninterpreted and can represent anything you like.

Throws:

`InterruptedException` - if the current thread is interrupted

tryAcquireSharedNanos

```
public final boolean tryAcquireSharedNanos(int arg,
                                           long nanosTimeout)
                                    throws InterruptedException
```

Attempts to acquire in shared mode, aborting if interrupted, and failing if the given timeout elapses. Implemented by first checking interrupted status, then invoking at least once `tryAcquireShared(int)`, returning on success. Otherwise, the thread is queued, possibly repeatedly blocking and unblocking, invoking `tryAcquireShared(int)` until success or the thread is interrupted or the timeout elapses.

Parameters:

`arg` - the acquire argument. This value is conveyed to `tryAcquireShared(int)` but is otherwise uninterpreted and can represent anything you like.

`nanosTimeout` - the maximum number of nanoseconds to wait

Returns:

`true` if acquired; `false` if timed out

Throws:

`InterruptedException` - if the current thread is interrupted

releaseShared

```
public final boolean releaseShared(int arg)
```

Releases in shared mode. Implemented by unblocking one or more threads if `tryReleaseShared(int)` returns true.

Parameters:

`arg` - the release argument. This value is conveyed to `tryReleaseShared(int)` but is otherwise uninterpreted and can represent anything you like.

Returns:

the value returned from `tryReleaseShared(int)`

hasQueuedThreads

```
public final boolean hasQueuedThreads()
```

Queries whether any threads are waiting to acquire. Note that because cancellations due to interrupts and timeouts may occur at any time, a `true` return does not guarantee that any other thread will ever acquire.

Returns:

`true` if there may be other threads waiting to acquire

hasContended

```
public final boolean hasContended()
```

Queries whether any threads have ever contended to acquire this synchronizer; that is, if an acquire method has ever blocked.

In this implementation, this operation returns in constant time.

Returns:

`true` if there has ever been contention

getFirstQueuedThread

```
public final Thread getFirstQueuedThread()
```

Returns the first (longest-waiting) thread in the queue, or `null` if no threads are currently queued.

In this implementation, this operation normally returns in constant time, but may iterate upon contention if other threads are concurrently modifying the queue.

Returns:

the first (longest-waiting) thread in the queue, or `null` if no threads are currently queued

isQueued

```
public final boolean isQueued(Thread thread)
```

Returns true if the given thread is currently queued.

This implementation traverses the queue to determine presence of the given thread.

Parameters:

`thread` - the thread

Returns:

`true` if the given thread is on the queue

Throws:

`NullPointerException` - if the thread is null

hasQueuedPredecessors

```
public final boolean hasQueuedPredecessors()
```

Queries whether any threads have been waiting to acquire longer than the current thread.

An
invocation of this method is equivalent to (but may be more efficient than):

```
getFirstQueuedThread() != Thread.currentThread()
  && hasQueuedThreads()
```

Note that because cancellations due to interrupts and timeouts may occur at any time, a `true` return does not guarantee that some other thread will acquire before the current thread. Likewise, it is possible for another thread to win a race to enqueue after this method has returned `false`, due to the queue being empty.

This method is designed to be used by a fair synchronizer to avoid barging. Such a synchronizer's `tryAcquire(int)` method should return `false`, and its `tryAcquireShared(int)` method should return a negative value, if this method returns `true` (unless this is a reentrant acquire). For example, the `tryAcquire` method for a fair, reentrant, exclusive mode synchronizer might look like this:

```
protected boolean tryAcquire(int arg) {
  if (isHeldExclusively()) {
    // A reentrant acquire; increment hold count
    return true;
  } else if (hasQueuedPredecessors()) {
    return false;
  } else {
    // try to acquire normally
  }
}
```

Returns:

`true` if there is a queued thread preceding the current thread, and `false` if the current thread is at the head of the queue or the queue is empty

Since:

1.7

getQueueLength

```
public final int getQueueLength()
```

Returns an estimate of the number of threads waiting to acquire. The value is only an estimate because the number of threads may change dynamically while this method traverses internal data structures. This method is designed for use in monitoring system state, not for synchronization control.

Returns:

the estimated number of threads waiting to acquire

getQueuedThreads

```
public final Collection<Thread> getQueuedThreads()
```

Returns a collection containing threads that may be waiting to acquire. Because the actual set of threads may change dynamically while constructing this result, the returned collection is only a best-effort estimate. The elements of the returned collection are in no particular order. This method is designed to facilitate construction of subclasses that provide more extensive monitoring facilities.

Returns:

the collection of threads

getExclusiveQueuedThreads

```
public final Collection<Thread> getExclusiveQueuedThreads()
```

Returns a collection containing threads that may be waiting to acquire in exclusive mode. This has the same properties as `getQueuedThreads()` except that it only returns those threads waiting due to an exclusive acquire.

Returns:

the collection of threads

getSharedQueuedThreads

```
public final Collection<Thread> getSharedQueuedThreads()
```

Returns a collection containing threads that may be waiting to acquire in shared mode. This has the same properties as `getQueuedThreads()` except that it only returns those threads waiting due to a shared acquire.

Returns:

the collection of threads

toString

```
public String toString()
```

Returns a string identifying this synchronizer, as well as its state. The state, in brackets, includes the String `"State ="` followed by the current value of `getState()`, and either `"nonempty"` or `"empty"` depending on whether the queue is empty.

Overrides:

`toString` in class `Object`

Returns:

a string identifying this synchronizer, as well as its state

owns

```
public final boolean owns
(AbstractQueuedSynchronizer.ConditionObject condition)
```

Queries whether the given ConditionObject uses this synchronizer as its lock.

Parameters:

`condition` - the condition

Returns:

`true` if owned

Throws:

`NullPointerException` - if the condition is null

hasWaiters

```
public final boolean hasWaiters
(AbstractQueuedSynchronizer.ConditionObject condition)
```

Queries whether any threads are waiting on the given condition associated with this synchronizer. Note that because timeouts and interrupts may occur at any time, a `true` return does not guarantee that a future `signal` will awaken any threads. This method is designed primarily for use in monitoring of the system state.

Parameters:

`condition` - the condition

Returns:

`true` if there are any waiting threads

Throws:

`IllegalMonitorStateException` - if exclusive synchronization is not held

`IllegalArgumentException` - if the given condition is not associated with this synchronizer

`NullPointerException` - if the condition is null

getWaitQueueLength

```
public final int getWaitQueueLength
(AbstractQueuedSynchronizer.ConditionObject condition)
```

Returns an estimate of the number of threads waiting on the given condition associated with this synchronizer. Note that because timeouts and interrupts may occur at any time, the estimate serves only as an upper bound on the actual number of waiters. This method is designed for use in monitoring system state, not for synchronization control.

Parameters:

`condition` - the condition

Returns:

the estimated number of waiting threads

Throws:

`IllegalMonitorStateException` - if exclusive synchronization is not held

`IllegalArgumentException` - if the given condition is not associated with this synchronizer

NullPointerException - if the condition is null

getWaitingThreads

```
public final Collection<Thread> getWaitingThreads
(AbstractQueuedSynchronizer.ConditionObject condition)
```

Returns a collection containing those threads that may be waiting on the given condition associated with this synchronizer. Because the actual set of threads may change dynamically while constructing this result, the returned collection is only a best-effort estimate. The elements of the returned collection are in no particular order.

Parameters:

condition - the condition

Returns:

the collection of threads

Throws:

IllegalMonitorStateException - if exclusive synchronization is not held

IllegalArgumentException - if the given condition is not associated with this synchronizer

NullPointerException - if the condition is null

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.